CADT
IDT
វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology
DEPARTMENT OF COMPUTER SCIENCE

Database Systems

05 Data Manipulation Language (DML)


Kak Soky, PhD

soky.kak@cadt.edu.kh

085683680

Objective

- To be familiar with SQL Language especially with Data Manipulation Language (DML).

INSERT INTO

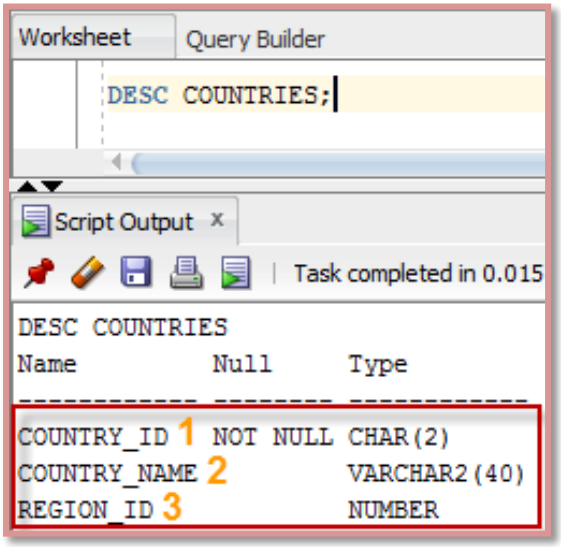
INSERT INTO statement used to insert rows in a table.

INSERT INTO Basic Syntax

INSERT INTO TableName
VALUES (Data Values**);**

The data values are listed in the order in which the columns appear in the table, separated by commas.

Note: use "**DESC** table-name" statement to see the columns order in table definition.



```
insert into COUNTRIES
values('PS','Palestine',4);
```

Script Output | Task completed in 1.284 seconds

```
1 rows inserted.
```

INSERT INTO With Specific Columns Syntax

INSERT INTO TableName **(**Columns Names**)**
VALUES (Data Values**);**

```
insert into EMPLOYEES(EMPLOYEE_ID,FIRST_NAME,LAST_NAME,EMAIL,JOB_ID,HIRE_DATE)
values (207,'Haneen','El-Masry','hmasry@iugaza.edu.ps','IT_PROG','5-NOV-14');
```

Script Output | Task completed in 0.127 seconds

```
1 rows inserted.
```

SELECT

The statement that is used to retrieve data from a database is called a **query**. In SQL the **SELECT** statement is used to specify queries.

SELECT Syntax

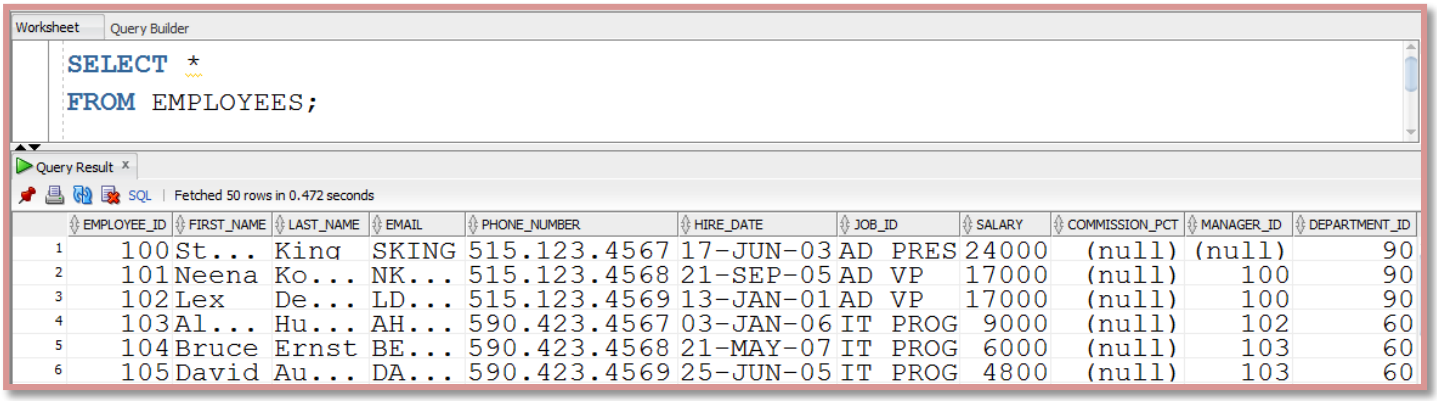
SELECT Attribute List
FROM Table List
[**WHERE** condition]
[**GROUP BY** grouping attributes
[**HAVING** <group selection condition>]]
[**ORDER BY** Columns || aliases || columns numbers]

Attribute List:

It is a list of the attributes we want to retrieve. It can be:

- ❖ * It is a fast alternative to all columns names.

Q1: Retrieve all the employees.



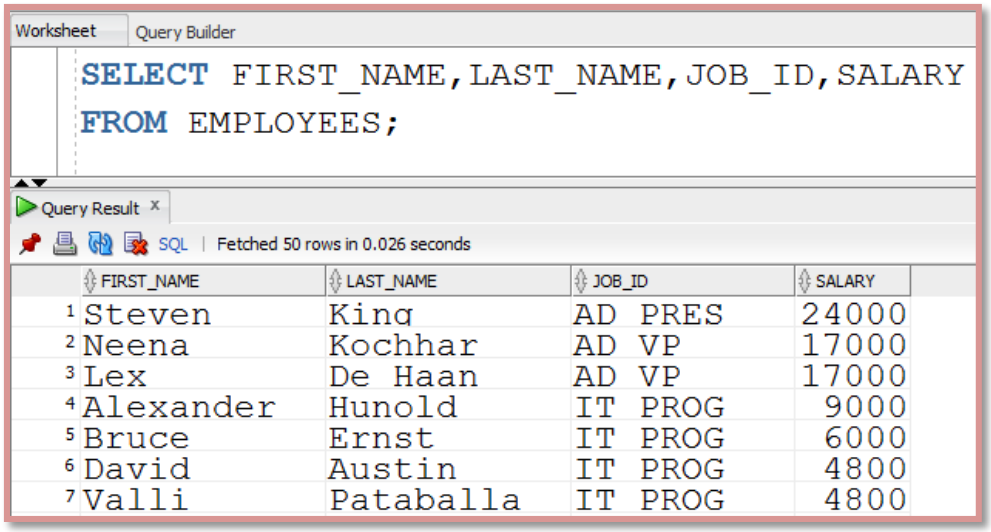
```
SELECT *
FROM EMPLOYEES;
```

Query Result — Fetched 50 rows in 0.472 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	St...	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Ko...	NK...	515.123.4568	21-SEP-05	AD_VP	17000	(null)	100	90
3	102	Lex	De...	LD...	515.123.4569	13-JAN-01	AD_VP	17000	(null)	100	90
4	103	Al...	Hu...	AH...	590.423.4567	03-JAN-06	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BE...	590.423.4568	21-MAY-07	IT_PROG	6000	(null)	103	60
6	105	David	Au...	DA...	590.423.4569	25-JUN-05	IT_PROG	4800	(null)	103	60

- ❖ **Specific Column(s).**

Q2: Retrieve the first name, last name, job id and salary for each Employee.



```
SELECT FIRST_NAME,LAST_NAME,JOB_ID,SALARY
FROM EMPLOYEES;
```

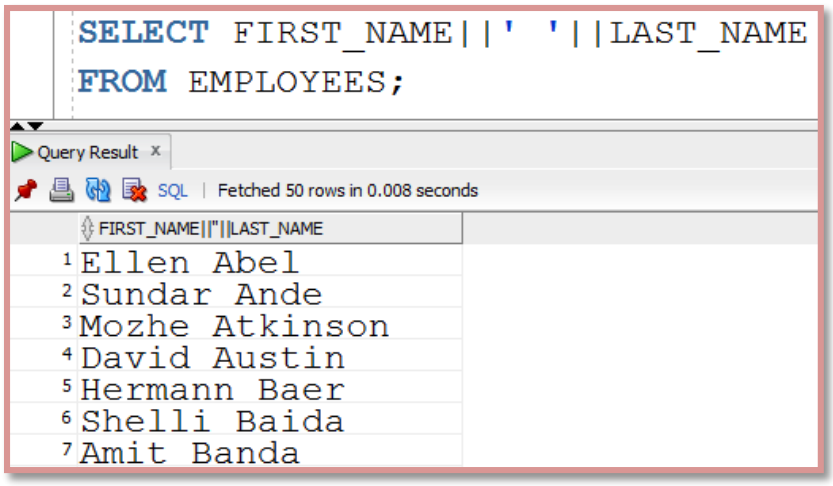
Query Result — Fetched 50 rows in 0.026 seconds

	FIRST_NAME	LAST_NAME	JOB_ID	SALARY
1	Steven	King	AD_PRES	24000
2	Neena	Kochhar	AD_VP	17000
3	Lex	De Haan	AD_VP	17000
4	Alexander	Hunold	IT_PROG	9000
5	Bruce	Ernst	IT_PROG	6000
6	David	Austin	IT_PROG	4800
7	Valli	Pataballa	IT_PROG	4800

Concatenation Operator

- ➢ Links columns or character strings to other columns.
- ➢ Is represented by two vertical bars (||).
- ➢ Creates a resultant column that is a character expression.

Q3: Retrieve the full name of each employee.

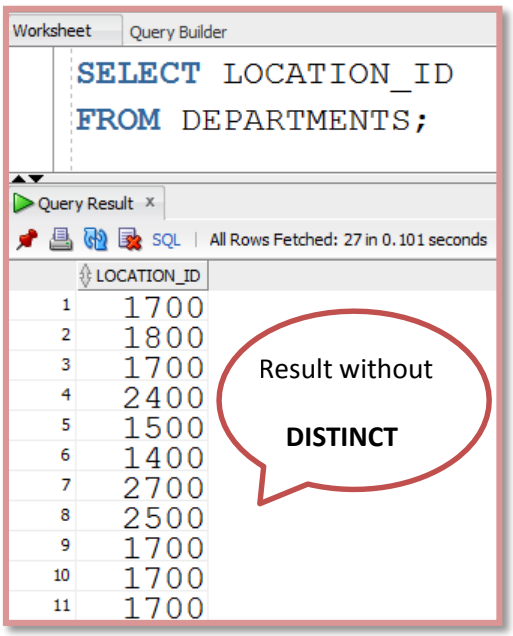


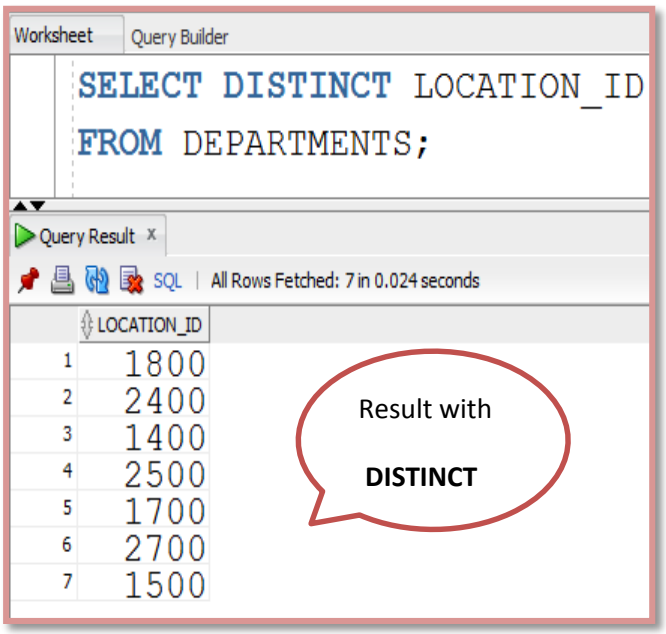
❖ **DISTINCT** COLUMN

In a table, some of the columns may contain duplicate values. If you want to list only the different (distinct) values in a table.

The **DISTINCT** keyword can be used to return only distinct (different) values.

Q4: Retrieve the location id of all departments.





❖ Arithmetic Expressions

Create expressions with numbers and date by using arithmetic operators.

Q5: Retrieve the Salary+300 for each employee.

```
SELECT FIRST_NAME,LAST_NAME,SALARY+300
FROM EMPLOYEES;
```

Query Result | SQL | Fetched 50 rows in 0.205 seconds

	FIRST_NAME	LAST_NAME	SALARY+300
1	Steven	King	24300
2	Neena	Kochhar	17300
3	Lex	De Haan	17300
4	Alexander	Hunold	9300
5	Bruce	Ernst	6300

Null Values

- A null is a value that is unavailable, unassigned, unknown, or inapplicable.
- It is **NOT** the same as a zero or a blank space.
- The result of any arithmetic expressions containing a null value is a **null value.**
- Use **NVL** function, which **converts a Null value into an actual specified value.**

Q6: Retrieve the total salary for each employee.

```
SELECT FIRST_NAME,SALARY,COMMISSION_PCT,SALARY+SALARY*COMMISSION_PCT
FROM EMPLOYEES;
```

Query Result | SQL | Fetched 100 rows in 0.061 seconds

	FIRST_NAME	SALARY	COMMISSION_PCT	SALARY+SALARY*COMMISSION_PCT
42	Trenna	3500	(null)	(null)
43	Curtis	3100	(null)	(null)
44	Randall	2600	(null)	(null)
45	Peter	2500	(null)	(null)
46	John	14000	0.4	19600
47	Karen	13500	0.3	17550
48	Alberto	12000	0.3	15600
49	Gerald	11000	0.3	14300
50	Eleni	10500	0.2	12600
51	Peter	10000	0.3	13000

```
SELECT FIRST_NAME,SALARY,COMMISSION_PCT,SALARY+SALARY*NVL(COMMISSION_PCT,0)
FROM EMPLOYEES;
```

Query Result | SQL | Fetched 100 rows in 0.028 seconds

	FIRST_NAME	SALARY	COMMISSION_PCT	SALARY+SALARY*NVL(COMMISSION_PCT,0)
42	Trenna	3500	(null)	3500
43	Curtis	3100	(null)	3100
44	Randall	2600	(null)	2600
45	Peter	2500	(null)	2500
46	John	14000	0.4	19600
47	Karen	13500	0.3	17550
48	Alberto	12000	0.3	15600
49	Gerald	11000	0.3	14300
50	Eleni	10500	0.2	12600
51	Peter	10000	0.3	13000

- **function (**[**DISTINCT**] column || ***)**

Aggregate (Group) Functions in SQL

Aggregate functions operate on sets of rows to give one result per group. These sets may comprise the entire table or the table split into groups.

Group Functions:

- COUNT.
- SUM.
- MAX.
- MIN.
- AVG.
- STDDEV.
- VARIANCE.

Notes:

- All group function **ignore** NULL values. To substitute a value for null values, use the **NVL** function.
- The data types for the functions argument may be CHAR, VARCHAR2, NUMBER, or DATE.

- The AVG, SUM, VARIANCE, and STDDEV functions can be used only with numeric data types.
- **DISTINCT** makes the function consider only non-duplicate values; ALL makes it consider every value, including duplicates. The default is ALL and therefore does not need to be specified.
- **COUNT(*)** returns the number of rows in a table that satisfy the criteria of the SELECT statement, including **duplicate rows and rows containing null values** in any of the columns.

Q7: Retrieve the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
SELECT SUM(SALARY),MAX(SALARY),MIN(SALARY),AVG(SALARY)
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.086 seconds

	SUM(SALARY)	MAX(SALARY)	MIN(SALARY)	AVG(SALARY)
1	691416	24000	2100	6461.8317...

Q8: Retrieve the total number of employees in the company.

```
SELECT COUNT(EMPLOYEE_ID)
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.005 seconds

	COUNT(EMPLOYEE_ID)
1	107

Q9: Retrieve the number of employees who can earn a commission.

```
SELECT COUNT (COMMISSION_PCT)
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.003 seconds

	COUNT(COMMISSION_PCT)
1	35

Ignore NULL values but count duplicate values.

Q10: Count the number of distinct salary values in the database.

```
SELECT COUNT (DISTINCT SALARY)
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.028 seconds

	COUNT(DISTINCTSALARY)
1	58

Q11: Retrieve the average of the Commissions of the employees.

```
SELECT AVG(COMMISSION_PCT)
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.12 seconds

	AVG(COMMISSION_PCT)
1	0.22285714285714285714285714

avg ignores NULL values, so the result is the average of the commissions of the employees who earn commission not of All employees.

```
SELECT AVG(NVL(COMMISSION_PCT,0))
FROM EMPLOYEES;
```

Query Result | SQL | All Rows Fetched: 1 in 0.003 seconds

	AVG(NVL(COMMISSION_PCT,0))
1	0.072897196261682242990654205607476

Replace NULL values with 0, so the result is the average of the commissions of all employees.

Column Alias

- Renames a column heading.
- Useful with calculations and concatenation.
- Immediately follows the column name (There can also be the optional **AS** keyword between the column name and alias).
- Requires double quotation marks if it contains spaces or special characters or if it is case sensitive.

```
SELECT FIRST_NAME FNAME,LAST_NAME AS LNAME,
SALARY+SALARY*NVL(COMMISSION_PCT,0)"Total Salary"
FROM EMPLOYEES;
```

Query Result | SQL | Fetched 50 rows in 0.032 seconds

	FNAME	LNAME	Total Salary
1	Steven	King	24000
2	Neena	Kochhar	17000
3	Lex	De Haan	17000
4	Alexander	Hunold	9000
5	Bruce	Ernst	6000
6	David	Austin	4800
7	Valli	Pataballa	4800

```
SELECT FIRST_NAME||' '||LAST_NAME "Full Name"
FROM EMPLOYEES;
```

Query Result | SQL | Fetched 50 rows in 0.008 seconds

	Full Name
1	Ellen Abel
2	Sundar Ande
3	Mozhe Atkinson
4	David Austin
5	Hermann Baer
6	Shelli Baida

Table List

It can be one table as all previous queries or a joined table.

Joined Table

A joined table is a table derived from two or more tables according to the rules of the particular join type.

Join Types

1. **CROSS JOIN**

CROSS JOIN Syntax	
Table1 **CROSS JOIN** Table2	OR
Table1**,** Table2	

It produces the cross-product of two tables. The joined table will contain a row for every possible combination of rows from Table1 to Table2 consisting of all columns in Table1 followed by all columns in Table2. If the tables have N and M rows respectively, the joined table will have N * M rows.

```
SELECT *
FROM REGIONS CROSS JOIN COUNTRIES;
```

Query Result

SQL | Fetching next 50 rows in 1.399 seconds

	REGION_ID	REGION_NAME	COUNTRY_ID	COUNTRY_NAME	REGION_ID_1
1	1	Europe	AR	Argentina	2
2	1	Europe	AU	Australia	3
3	1	Europe	BE	Belgium	1
4	1	Europe	BR	Brazil	2
5	1	Europe	CA	Canada	2
6	1	Europe	CH	Switzerland	1
7	1	Europe	CN	China	3
8	1	Europe	DE	Germany	1
9	1	Europe	DK	Denmark	1
10	1	Europe	EG	Egypt	4
11	1	Europe	FR	France	1

Table Alias

- Renames a table in the query.
- Useful with join.
- Immediately follows the table name.
- Requires double quotation marks if it contains spaces or special characters or if it is case sensitive.
- You must alias the tables if you join the table with itself.

Note:

If the joined table contains two attributes with the same name, in the attribute list the relation (table) name or alias prefixed to the attribute name and separating the two by a period (.).

```
SELECT R.REGION_ID,REGION_NAME,COUNTRY_NAME
FROM REGIONS R,COUNTRIES N;
```

Query Result

SQL | Fetched 50 rows in 0.027 seconds

	REGION_ID	REGION_NAME	COUNTRY_NAME
1	1	Europe	Argentina
2	1	Europe	Australia
3	1	Europe	Belgium
4	1	Europe	Brazil
5	1	Europe	Canada
6	1	Europe	Switzerland
7	1	Europe	China
8	1	Europe	Germany
9	1	Europe	Denmark
10	1	Europe	Egypt
11	1	Europe	France

2. INNER JOIN

INNER JOIN Syntax
Table1 [**INNER**] **JOIN** Table2 Join_Condition

For each row R1 of Table1, the joined table has a row for each row in Table2 that satisfies the join condition with R1.

Join Condition

It can be:

- **ON (**boolean_expression**)**

The output of JION with an ON condition has all columns of the two tables.

```
SELECT *
FROM REGIONS R join COUNTRIES N ON (R.REGIOn_ID=N.REGION_ID);
```

Query Result | SQL | All Rows Fetched: 25 in 0.059 seconds

	REGION_ID	REGION_NAME	COUNTRY_ID	COUNTRY_NAME	REGION_ID_1
1	1	Europe	NL	Netherlands	1
2	1	Europe	FR	France	1
3	1	Europe	UK	United Kingdom	1
4	1	Europe	DK	Denmark	1
5	1	Europe	BE	Belgium	1
6	1	Europe	CH	Switzerland	1
7	1	Europe	IT	Italy	1
8	1	Europe	DE	Germany	1
9	2	Americas	US	United States of America	2
10	2	Americas	CA	Canada	2

Q12: Retrieve department Id and department name for each employee.

```
SELECT FIRST_NAME,D.DEPARTMENT_ID,DEPARTMENT_NAME
FROM EMPLOYEES E JOIN DEPARTMENTS D ON(E.DEPARTMENT_ID=D.DEPARTMENT_ID);
```

Query Result | SQL | Fetched 50 rows in 0.345 seconds

	FIRST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Jennifer	10	Administration
2	Pat	20	Marketing
3	Michael	20	Marketing
4	Sigal	30	Purchasing
5	Karen	30	Purchasing
6	Shelli	30	Purchasing
7	Den	30	Purchasing
8	Alexander	30	Purchasing

D.DEPARTMENT_ID because DEPARTMENT_ID appears twice in the joined table.

Q13: Retrieve the full name of each employee and the full name of his manager.

```
SELECT EMP.FIRST_NAME||' '||EMP.LAST_NAME "EMPLOYEE NAME",
       MANAGER.FIRST_NAME||' '||MANAGER.LAST_NAME "MANAGER NAME"
FROM EMPLOYEES EMP JOIN EMPLOYEES MANAGER ON(EMP.MANAGER_ID=MANAGER.EMPLOYEE_ID);
```

Query Result | SQL | Fetched 50 rows in 0.014 seconds

	EMPLOYEE NAME	MANAGER NAME
1	Eleni Zlotkey	Steven King
2	Matthew Weiss	Steven King
3	Shanta Vollman	Steven King
4	John Russell	Steven King
5	Den Raphaely	Steven King
6	Karen Partners	Steven King
7	Kevin Mourgos	Steven King
8	Neena Kochhar	Steven King
9	Payam Kaufling	Steven King

Here, you must alias the tables because the join is **self-join**.

- **USING (** join column list **)**

It takes a comma-separated list of equated column names from the two tables.

The output of JOIN USING has one column for each of the equated pairs of input columns, followed by the remaining columns from each table.

```
SELECT *
FROM REGIONS inner join COUNTRIES USING (REGION_ID);
```

Query Result | SQL | All Rows Fetched: 25 in 0.166 seconds

	REGION_ID	REGION_NAME	COUNTRY_ID	COUNTRY_NAME
1	1	Europe	NL	Netherlands
2	1	Europe	FR	France
3	1	Europe	UK	United Kingdom
4	1	Europe	DK	Denmark
5	1	Europe	BE	Belgium
6	1	Europe	CH	Switzerland
7	1	Europe	IT	Italy
8	1	Europe	DE	Germany
9	2	Americas	US	United States of America
10	2	Americas	CA	Canada

Q14: Retrieve region id and region name for each country.

```
SELECT COUNTRY_NAME,REGION_ID,REGION_NAME
FROM COUNTRIES N JOIN REGIONS R USING(REGION_ID);
```

Query Result | SQL | All Rows Fetched: 25 in 0.068 seconds

	COUNTRY_NAME	REGION_ID	REGION_NAME
1	Netherlands	1	Europe
2	France	1	Europe
3	United Kingdom	1	Europe
4	Denmark	1	Europe
5	Belgium	1	Europe
6	Switzerland	1	Europe
7	Italy	1	Europe
8	Germany	1	Europe
9	United States...	2	Americas
10	Canada	2	Americas

Q15: Retrieve country name for each department.

```
SELECT DEPARTMENT_NAME, COUNTRY_NAME
FROM DEPARTMENTS JOIN LOCATIONS USING(LOCATION_ID)JOIN COUNTRIES USING(COUNTRY_ID);
```

Query Result | SQL | All Rows Fetched: 27 in 1.503 seconds

	DEPARTMENT_NAME	COUNTRY_NAME
1	IT	United States of America
2	Shipping	United States of America
3	Administration	United States of America
4	Purchasing	United States of America
5	Executive	United States of America
6	Finance	United States of America
7	Accounting	United States of America

3. NATURAL INNER JOIN

NATURAL INNER JOIN Syntax

Table1 **NATURAL** [**INNER**] **JOIN** Table2

It forms a USING list consisting of all column names that appear in both input tables. As with USING, these columns appear only once in the output table.

```
SELECT *
FROM REGIONS NATURAL JOIN COUNTRIES;
```

Query Result | SQL | All Rows Fetched: 25 in 4.188 seconds

	REGION_ID	REGION_NAME	COUNTRY_ID	COUNTRY_NAME
1	1	Europe	NL	Netherlands
2	1	Europe	FR	France
3	1	Europe	UK	United Kingdom
4	1	Europe	DK	Denmark
5	1	Europe	BE	Belgium
6	1	Europe	CH	Switzerland
7	1	Europe	IT	Italy
8	1	Europe	DE	Germany

Q16: Retrieve job id and job name for each employee.

```
SELECT FIRST_NAME,JOB_ID,JOB_TITLE
FROM EMPLOYEES NATURAL JOIN JOBS;
```

Query Result | SQL | Fetched 50 rows in 0.343 seconds

	FIRST_NAME	JOB_ID	JOB_TITLE
1	William	AC_ACCOUNT	Public Accountant
2	Shelley	AC_MGR	Accounting Manager
3	Jennifer	AD_ASST	Administration As...
4	Steven	AD_PRES	President
5	Neena	AD_VP	Administration Vi...
6	Lex	AD_VP	Administration Vi...
7	Jose Manuel	FI_ACCOUNT	Accountant
8	Ismael	FI_ACCOUNT	Accountant

4. LEFT OUTER JOIN

LEFT OUTER JOIN Syntax
Table1 **LEFT** [**OUTER**] **JOIN** Table2 Join_Condition

First, an inner join is performed. Then, for each row in Table1 that does not satisfy the join condition with any row in Table2, a joined row is added with NULL values in columns of Table2.

Thus, the joined table always has at least one row for each row in Table1.

Note: LEFT OUTER JOIN join_condition can be ON or USING condition.

```
SELECT *
FROM COUNTRIES N LEFT OUTER JOIN LOCATIONS L ON (N.COUNTRY_ID=L.COUNTRY_ID);
```

Query Result | SQL | All Rows Fetched: 34 in 0.022 seconds

	COUNTRY_ID	COUNTRY_NAME	REGION_ID	LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID_1
20	CH	Switzer...	1	2900	20 Rue ...	1730	Geneva	Geneve	CH
21	CH	Switzer...	1	3000	Murtens...	3095	Bern	BE	CH
22	NL	Netherl...	1	3100	Pieter ...	3029SK	Utrecht	Utrecht	NL
23	MX	Mexico	2	3200	Mariano...	11932	Mexico...	Distrit...	MX
24	ML	Malaysia	3	(null)	(null)	(null)	(null)	(null)	(null)
25	AR	Argentina	2	(null)	(null)	(null)	(null)	(null)	(null)
26	IL	Israel	4	(null)	(null)	(null)	(null)	(null)	(null)
27	NG	Nigeria	4	(null)	(null)	(null)	(null)	(null)	(null)
28	EG	Egypt	4	(null)	(null)	(null)	(null)	(null)	(null)
29	KW	Kuwait	4	(null)	(null)	(null)	(null)	(null)	(null)

```
SELECT *
FROM COUNTRIES N LEFT JOIN LOCATIONS L USING (COUNTRY_ID);
```

Query Result | SQL | All Rows Fetched: 34 in 0.013 seconds

	COUNTRY_ID	COUNTRY_NAME	REGION_ID	LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE
20	CH	Switz...	1	2900	20 Rue...	1730	Geneva	Geneve
21	CH	Switz...	1	3000	Murten...	3095	Bern	BE
22	NL	Nethe...	1	3100	Pieter...	3029SK	Utrecht	Utrecht
23	MX	Mexico	2	3200	Marian...	11932	Mexico...	Distri...
24	ML	Malaysia	3	(null)	(null)	(null)	(null)	(null)
25	AR	Argen...	2	(null)	(null)	(null)	(null)	(null)
26	IL	Israel	4	(null)	(null)	(null)	(null)	(null)
27	NG	Nigeria	4	(null)	(null)	(null)	(null)	(null)
28	EG	Egypt	4	(null)	(null)	(null)	(null)	(null)
29	KW	Kuwait	4	(null)	(null)	(null)	(null)	(null)

5. NATURAL LEFT OUTER JOIN

NATURAL LEFT OUTER JOIN Syntax

Table1 **NATURAL LEFT**[**OUTER**] **JOIN** Table2

It forms a USING list consisting of all column names that appear in both input tables. As with USING, these columns appear only once in the output table.

```
SELECT *
FROM COUNTRIES N NATURAL LEFT JOIN LOCATIONS L;
```

Query Result | SQL | All Rows Fetched: 34 in 2.528 seconds

	COUNTRY_ID	COUNTRY_NAME	REGION_ID	LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE
20	CH	Switze...	1	2900	20 Rue d...	1730	Geneva	Geneve
21	CH	Switze...	1	3000	Murtenst...	3095	Bern	BE
22	NL	Nether...	1	3100	Pieter B...	3029SK	Utrecht	Utrecht
23	MX	Mexico	2	3200	Mariano ...	11932	Mexico ...	Distrito Fede...
24	ML	Malaysia	3	(null)	(null)	(null)	(null)	(null)
25	AR	Argentina	2	(null)	(null)	(null)	(null)	(null)
26	IL	Israel	4	(null)	(null)	(null)	(null)	(null)
27	NG	Nigeria	4	(null)	(null)	(null)	(null)	(null)
28	EG	Egypt	4	(null)	(null)	(null)	(null)	(null)
29	KW	Kuwait	4	(null)	(null)	(null)	(null)	(null)

6. RIGHT OUTER JOIN

RIGHT OUTER JOIN Syntax

Table1 **RIGHT** [**OUTER**] **JOIN** Table2 Join_Condition

First, an inner join is performed. Then, for each row in Table2 that does not satisfy the join condition with any row in Table1, a joined row is added with null values in columns of Table1.

This is the converse of a left join. The result table will always have a row for each row in Table2.

Note: RIGHT OUTER JOIN join_condition can be ON or USING condition.

```
SELECT *
FROM DEPARTMENTS D RIGHT JOIN LOCATIONS L ON(D.LOCATION_ID=L.LOCATION_ID);
```

Query Result | SQL | All Rows Fetched: 43 in 0.035 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID	LOCATION_ID_1	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
1	(null)	(null)	(null)	(null)	10001297	...	00989	Roma	(null)	IT
2	(null)	(null)	(null)	(null)	110093091	...	10934	Venice	(null)	IT
3	(null)	(null)	(null)	(null)	12002017	...	1689	Tokyo	Tokyo...	JP
4	(null)	(null)	(null)	(null)	13009450	...	6823	Hir...	(null)	JP
5	60	IT	103	1400	14002014	...	26192	Sou...	Texas	US

```
SELECT *
FROM DEPARTMENTS D RIGHT JOIN LOCATIONS L USING(LOCATION_ID);
```

Query Result | SQL | All Rows Fetched: 43 in 1.095 seconds

	LOCATION_ID	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
1	1000	(null)	(null)	(null)	1297 Via Cola...	00989	Roma	(null)	IT
2	1100	(null)	(null)	(null)	93091 Calle d...	10934	Venice	(null)	IT
3	1200	(null)	(null)	(null)	2017 Shinjuku-ku	1689	Tokyo	Tokyo...	JP
4	1300	(null)	(null)	(null)	9450 Kamiya-cho	6823	Hiroshima	(null)	JP
5	1400	60	IT	103	2014 Jabberwo...	26192	Southlake	Texas	US

7. NATURAL RIGHT OUTER JOIN

NATURAL RIGHT OUTER JOIN Syntax

Table1 **NATURAL RIGHT**[**OUTER**] **JOIN** Table2

It forms a USING list consisting of all column names that appear in both input tables. As with USING, these columns appear only once in the output table.

```
SELECT *
FROM DEPARTMENTS D NATURAL RIGHT OUTER JOIN LOCATIONS L;
```

Query Result | SQL | All Rows Fetched: 43 in 0.815 seconds

	LOCATION_ID	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
1	1000	(null)	(null)	(null)	1297 ...	00989	Roma	(null)	IT
2	1100	(null)	(null)	(null)	93091...	10934	Venice	(null)	IT
3	1200	(null)	(null)	(null)	2017 ...	1689	Tokyo	Tokyo ...	JP
4	1300	(null)	(null)	(null)	9450 ...	6823	Hiro...	(null)	JP
5	1400	60	IT	103	2014 ...	26192	Sout...	Texas	US

8. FULL OUTER JOIN

FULL OUTER JOIN Syntax

Table1 **FULL** [**OUTER**] **JOIN** Table2 Join_Condition

First, an inner join is performed. Then, for each row in Table1 that does not satisfy the join condition with any row in Table2, a joined row is added with NULL values in columns of Table2.

Also, for each row of Table2 that does not satisfy the join condition with any row in Table1, a joined row with NULL values in the columns of Table1 is added.

Note: FULL OUTER JOIN join_condition can be ON or USING condition.

```
SELECT *
FROM EMPLOYEES E FULL JOIN JOB_HISTORY J
    ON (E.JOB_ID=J.JOB_ID
    AND E.EMPLOYEE_ID=J.EMPLOYEE_ID
    AND E.DEPARTMENT_ID=J.DEPARTMENT_ID);
```

Query Result | All Rows Fetched: 116 in 0.597 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE...	HIRE_DATE	JOB_ID	SALARY	COMMISSION...	MANAGE...	DEPARTMENT_ID	EMPLOYEE_ID_1	START_DATE	END_DATE	JOB_ID_1	DEPARTMENT_ID_1
102	201	Michael	Hartstein	MHAR...	515....	17-FEB-04	MK_MAN	13000	(null)	100	20	(null)	(null)	(null)	(null)	(null)
103	202	Pat	Fay	PFAY	603....	17-AUG-05	MK_REP	6000	(null)	201	20	(null)	(null)	(null)	(null)	(null)
104	203	Susan	Mavris	SMAVRIS	515....	07-JUN-02	HR_REP	6500	(null)	101	40	(null)	(null)	(null)	(null)	(null)
105	204	Hermann	Baer	HBAER	515....	07-JUN-02	PR_REP	10000	(null)	101	70	(null)	(null)	(null)	(null)	(null)
106	205	Shelley	Higgins	SHIG...	515....	07-JUN-02	AC_MGR	12008	(null)	101	110	(null)	(null)	(null)	(null)	(null)
107	206	William	Gietz	WGIETZ	515....	07-JUN-02	AC_...	8300	(null)	205	110	(null)	(null)	(null)	(null)	(null)
108	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	102	13-JAN-01	24-JU...	IT_PROG	60
109	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	101	21-SEP-97	27-OC...	AC_AC...	110
110	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	122	01-JAN-07	31-DE...	ST_CLERK	50
111	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	200	17-SEP-95	17-JU...	AD_ASST	90

```
SELECT *
FROM EMPLOYEES E FULL JOIN JOB_HISTORY J USING (JOB_ID,EMPLOYEE_ID,DEPARTMENT_ID);
```

Query Result | All Rows Fetched: 116 in 0.128 seconds

	JOB_ID	EMPLOYEE_ID	DEPARTMENT_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	SALARY	COMMISSION_PCT	MANAGER_ID	START_DATE	END_DATE
102	MK_MAN	201	20	Mic...	Har...	MH...	515.123.5555	17-FEB-04	13000	(null)	100	(null)	(null)
103	MK_REP	202	20	Pat	Fay	PFAY	603.123.6666	17-AUG-05	6000	(null)	201	(null)	(null)
104	HR_REP	203	40	Susan	Mavris	SM...	515.123.7777	07-JUN-02	6500	(null)	101	(null)	(null)
105	PR_REP	204	70	Her...	Baer	HBAER	515.123.8888	07-JUN-02	10000	(null)	101	(null)	(null)
106	AC_MGR	205	110	She...	Hig...	SH...	515.123.8080	07-JUN-02	12008	(null)	101	(null)	(null)
107	AC_A...	206	110	Wil...	Gietz	WG...	515.123.8181	07-JUN-02	8300	(null)	205	(null)	(null)
108	SA_MAN	176	80	(null)	(null)	(n...	(null)	(null)	(n...	(null)	(null)	01-...	31-...
109	AC_A...	101	110	(null)	(null)	(n...	(null)	(null)	(n...	(null)	(null)	21-...	27-...
110	AC_A...	200	90	(null)	(null)	(n...	(null)	(null)	(n...	(null)	(null)	01-...	31-...
111	ST_C...	114	50	(null)	(null)	(n...	(null)	(null)	(n...	(null)	(null)	24-...	31-...

9. NATURAL FULL OUTER JOIN

NATURAL FULL OUTER JOIN Syntax

Table1 **NATURAL FULL**[**OUTER**] **JOIN** Table2

It forms a USING list consisting of all column names that appear in both input tables. As with USING, these columns appear only once in the output table.

```
SELECT *
FROM EMPLOYEES NATURAL FULL JOIN JOB_HISTORY;
```

Query Result | SQL | All Rows Fetched: 116 in 0.067 seconds

	EMPLOYEE_ID	JOB_ID	DEPARTMENT_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	SALARY	COMMISSION_PCT	MANAGER_ID	START_DATE	END_DATE
102	201	MK_MAN	20	Mic...	Hart...	MHARTSTE	515.12...	17-FEB-04	13000	(null)	100	(null)	(null)
103	202	MK_REP	20	Pat	Fay	PFAY	603.12...	17-AUG-05	6000	(null)	201	(null)	(null)
104	203	HR_REP	40	Susan	Mavris	SMAVRIS	515.12...	07-JUN-02	6500	(null)	101	(null)	(null)
105	204	PR_REP	70	Her...	Baer	HBAER	515.12...	07-JUN-02	10000	(null)	101	(null)	(null)
106	205	AC_MGR	110	She...	Higgins	SHIGGINS	515.12...	07-JUN-02	12008	(null)	101	(null)	(null)
107	206	AC_A...	110	Wil...	Gietz	WGIETZ	515.12...	07-JUN-02	8300	(null)	205	(null)	(null)
108	200	AC_A...	90	(null)	(null)	(null)	(null)	(null)	(n...	(null)	(null)	01-...	31-...
109	102	IT_PROG	60	(null)	(null)	(null)	(null)	(null)	(n...	(null)	(null)	13-...	24-...
110	101	AC_A...	110	(null)	(null)	(null)	(null)	(null)	(n...	(null)	(null)	21-...	27-...
111	101	AC_MGR	110	(null)	(null)	(null)	(null)	(null)	(n...	(null)	(null)	28-...	15-...
112	200	AD_ASST	90	(null)	(null)	(null)	(null)	(null)	(n	(null)	(null)	17-	17-

Restricting Data

In most cases, we query the database to get some specific data from a table and not the whole table; therefore, we need a technique to restrict the data retrieved by SELECT statement.

Restricting the rows that are returned by a query can be done by using the optional [**WHERE** clause].

Comparison operators:

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN … AND …	Between two values (inclusive)
LIKE	Match a character pattern: **%**: any number of any characters. **_**: one character.
IS NULL	Is a null value
IN(set)	Match any of a list of values
ANY(set)	Compare to any value in a set
ALL(set)	Compare to all values in a set

Notes:

- When you deal with character strings or date values, you must enclosed them by single quotation marks (' ').
- Character values are case-sensitive, and date values are format-sensitive.
- The default date format is DD-MON-RR.

Q17: Retrieve all employees who are working in department 30.

```
SELECT FIRST_NAME,LAST_NAME,DEPARTMENT_ID
FROM EMPLOYEES
WHERE DEPARTMENT_ID=30;
```

Query Result | SQL | All Rows Fetched: 6 in 1.068 seconds

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Den	Raphaely	30
2	Alexander	Khoo	30
3	Shelli	Baida	30
4	Sigal	Tobias	30
5	Guy	Himuro	30
6	Karen	Colmenares	30

Q18: Retrieve all employees whose first name is “David”.

```
SELECT FIRST_NAME,LAST_NAME
FROM EMPLOYEES
WHERE FIRST_NAME='David';
```

Query Result | SQL | All Rows Fetched: 3 in 0.081 seconds

	FIRST_NAME	LAST_NAME
1	David	Austin
2	David	Bernstein
3	David	Lee

Q19: Retrieve all employees who are hired on 7/6/2002.

```
SELECT FIRST_NAME,LAST_NAME,HIRE_DATE
FROM EMPLOYEES
WHERE HIRE_DATE='07-JUN-02';
```

Query Result | SQL | All Rows Fetched: 4 in 0.106 seconds

	FIRST_NAME	LAST_NAME	HIRE_DATE
1	Susan	Mavris	07-JUN-02
2	Hermann	Baer	07-JUN-02
3	Shelley	Higgins	07-JUN-02
4	William	Gietz	07-JUN-02

Q20: Retrieve all employees whose first name starts with “A”.

```
SELECT FIRST_NAME,LAST_NAME
FROM EMPLOYEES
WHERE FIRST_NAME LIKE 'A%';
```

Query Result | SQL | All Rows Fetched: 10 in 0.039 seconds

	FIRST_NAME	LAST_NAME
1	Amit	Banda
2	Alexis	Bull
3	Anthony	Cabrio
4	Alberto	Errazuriz
5	Adam	Fripp
6	Alexander	Hunold
7	Alyssa	Hutton
8	Alexander	Khoo
9	Allan	McEwen
10	Alana	Walsh

Q21: Retrieve all employees whose first name starts with “A” and the third letter is “e”.

```
SELECT FIRST_NAME,LAST_NAME
FROM EMPLOYEES
WHERE FIRST_NAME LIKE 'A_e%';
```

Query Result | SQL | All Rows Fetched: 3 in 0.003 seconds

	FIRST_NAME	LAST_NAME
1	Alexis	Bull
2	Alexander	Hunold
3	Alexander	Khoo

Q22: Retrieve the names of all employees whose salary between 5000 and 6000.

```
SELECT FIRST_NAME,LAST_NAME,SALARY
FROM EMPLOYEES
WHERE SALARY BETWEEN 5000 AND 6000;
```

Query Result | SQL | All Rows Fetched: 3 in 0.114 seconds

	FIRST_NAME	LAST_NAME	SALARY
1	Bruce	Ernst	6000
2	Kevin	Mourgos	5800
3	Pat	Fay	6000

Q23: Retrieve all employees who are working in departments: 60, 90, or 100.

```
SELECT FIRST_NAME,LAST_NAME,DEPARTMENT_ID
FROM EMPLOYEES
WHERE DEPARTMENT_ID IN (60,90,100);
```

Query Result | All Rows Fetched: 14 in 0.148 seconds

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	David	Austin	60
2	John	Chen	100
3	Lex	De Haan	90
4	Bruce	Ernst	60
5	Daniel	Faviet	100
6	Nancy	Greenberg	100
7	Alexander	Hunold	60
8	Steven	King	90
9	Neena	Kochhar	90
10	Diana	Lorentz	60
11	Valli	Pataballa	60
12	Luis	Popp	100
13	Ismael	Sciarra	100
14	Jose Manuel	Urman	100

Q24: Retrieve all employees who are working in departments 100 AND their salary is greater than ALL employees in department 60.

To solve this query, Firstly, we want to know the salaries of department 60.

```
SELECT DISTINCT SALARY
FROM EMPLOYEES
WHERE DEPARTMENT_ID=60;
```

Query Result | All Rows Fetched: 4 in 0.287 seconds

	SALARY
1	9000
2	4800
3	4200
4	6000

Then, find the employees in department 100 with their salary > all the values were retrieved from previous query.

```
SELECT FIRST_NAME,LAST_NAME,SALARY
FROM EMPLOYEES
WHERE DEPARTMENT_ID =100 AND SALARY > ALL (9000,4800,4200,6000);
```

Query Result | All Rows Fetched: 1 in 0.063 seconds

	FIRST_NAME	LAST_NAME	SALARY
1	Nancy	Greenberg	12008

In the next lab, we will learn how to solve queries like this query using one query.

Q25: Retrieve employee ID, employee last name, department ID and department name of all employees who are hired before 2004.

```
SELECT EMPLOYEE_ID,LAST_NAME,DEPARTMENT_ID,DEPARTMENT_NAME
FROM EMPLOYEES JOIN DEPARTMENTS USING (DEPARTMENT_ID)
WHERE HIRE_DATE < '1-JAN-04';
```

Query Result | SQL | All Rows Fetched: 14 in 0.521 seconds

	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	200	Whalen	10	Administration
2	115	Khoo	30	Purchasing
3	114	Raphaely	30	Purchasing
4	203	Mavris	40	Human Resources
5	122	Kaufling	50	Shipping
6	141	Rajs	50	Shipping
7	137	Ladwig	50	Shipping
8	204	Baer	70	Public Relations
9	102	De Haan	90	Executive
10	100	King	90	Executive
11	108	Greenberg	100	Finance
12	109	Faviet	100	Finance
13	205	Higgins	110	Accounting
14	206	Gietz	110	Accounting

Q26: Retrieve first name, last name, department name and city, for all employees whose salary is greater than 9000.

```
SELECT FIRST_NAME,LAST_NAME,DEPARTMENT_NAME,CITY
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS NATURAL JOIN LOCATIONS
WHERE SALARY > 9000;
```

Query Result | SQL | All Rows Fetched: 4 in 0.01 seconds

	FIRST_NAME	LAST_NAME	DEPARTMENT_NAME	CITY
1	Neena	Kochhar	Executive	Seattle
2	Lex	De Haan	Executive	Seattle
3	Peter	Tucker	Sales	Oxford
4	David	Bernstein	Sales	Oxford

Q27: Retrieve first name, last name, job id, job title, department id, and department name for all employees who work in Toronto City.

```
SELECT FIRST_NAME,LAST_NAME,E.JOB_ID,JOB_TITLE,DEPARTMENT_ID,DEPARTMENT_NAME
FROM EMPLOYEES E JOIN JOBS J ON (E.JOB_ID=J.JOB_ID)
     JOIN DEPARTMENTS USING(DEPARTMENT_ID)
     NATURAL JOIN LOCATIONS
WHERE CITY='Toronto';
```

Query Result | SQL | Fetched 50 rows in 0.015 seconds

	FIRST_NAME	LAST_NAME	JOB_ID	JOB_TITLE	DEPARTMENT_ID	DEPARTMENT_NAME
1	Jennifer	Whalen	AD_ASST	Administration Assistant	10	Administration
2	Pat	Fay	MK_REP	Marketing Representative	20	Marketing
3	Michael	Hartstein	MK_MAN	Marketing Manager	20	Marketing
4	Den	Raphaely	PU_MAN	Purchasing Manager	30	Purchasing

Q28: Retrieve the first name, last name of all employees that don't have manager.

```
SELECT FIRST_NAME,LAST_NAME
FROM EMPLOYEES
WHERE MANAGER_ID IS NULL;
```

Query Result | SQL | All Rows Fetched: 1 in 0.064 seconds

	FIRST_NAME	LAST_NAME
1	Steven	King

Q29: Retrieve the names and hire dates of all the employees who are hired before their managers, along with their managers' names and hire dates.

```
SELECT EMP.FIRST_NAME||' '||EMP.LAST_NAME "EMP FULL NAME",EMP.HIRE_DATE AS "EMP HIRE DATE",
       MGR.FIRST_NAME||' '||MGR.LAST_NAME "MANAGER FULL NAME",MGR.HIRE_DATE "MANAGER HIRE DATE"
FROM EMPLOYEES EMP, EMPLOYEES MGR
WHERE EMP.MANAGER_ID=MGR.EMPLOYEE_ID AND EMP.HIRE_DATE < MGR.HIRE_DATE;
```

Query Result | SQL | All Rows Fetched: 37 in 0.123 seconds

	EMP FULL NAME	EMP HIRE DATE	MANAGER FULL NAME	MANAGER HIRE DATE
1	Payam Kaufling	01-MAY-03	Steven King	17-JUN-03
2	Den Raphaely	07-DEC-02	Steven King	17-JUN-03
3	Lex De Haan	13-JAN-01	Steven King	17-JUN-03
4	Shelley Higgins	07-JUN-02	Neena Kochhar	21-SEP-05
5	Hermann Baer	07-JUN-02	Neena Kochhar	21-SEP-05
6	Susan Mavris	07-JUN-02	Neena Kochhar	21-SEP-05

Grouping

You can use the **GROUP BY** clause to divide the rows in a table into groups. You can then use the group functions to return summary information for each group.

Notes:

- Using a WHERE clause, you can exclude rows before dividing them into groups.
- You cannot use a column alias in the GROUP BY clause.
- When using the GROUP BY clause, make sure that all columns in the SELECT list that are not group functions are included in the GROUP BY clause.

Q30: For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
SELECT DEPARTMENT_ID, COUNT(EMPLOYEE_ID), AVG(SALARY)
FROM EMPLOYEES
GROUP BY DEPARTMENT_ID;
```

Query Result

SQL | All Rows Fetched: 12 in 1.598 seconds

	DEPARTMENT_ID	COUNT(EMPLOYEE_ID)	AVG(SALARY)
1	100	6	8601.3333333...
2	30	6	4150
3	(null)	1	7000
4	90	3	19333.333333...
5	20	2	9500
6	70	1	10000
7	110	2	10154
8	50	45	3475.5555555...
9	80	34	8955.8823529...
10	40	1	6500
11	60	5	5760
12	10	1	4400

Q31: For each job, retrieve its department id and the sum of the salary of the employees.

```
SELECT JOB_ID,DEPARTMENT_ID,SUM(SALARY)
FROM EMPLOYEES
GROUP BY JOB_ID, DEPARTMENT_ID;
```

Query Result

SQL | All Rows Fetched: 20 in 2.615 seconds

	JOB_ID	DEPARTMENT_ID	SUM(SALARY)
1	IT_PROG	60	28800
2	FI_MGR	100	12008
3	PU_MAN	30	11000
4	ST_MAN	50	36400

Restricting Group Results

You cannot use group functions in the WHERE clause. **HAVING** clause is used to specify the groups that are to be displayed, thus further restricting the groups on the basis of aggregate information.

HAVING provides a condition on the groups which necessarily involve an aggregate function.

Q32: For each department that its employee's average salary is greater than 8000, retrieve the average salary of its employees.

```
SELECT D.DEPARTMENT_ID,DEPARTMENT_NAME,AVG(SALARY)
FROM EMPLOYEES E,DEPARTMENTS D
WHERE E.DEPARTMENT_ID=D.DEPARTMENT_ID
GROUP BY D.DEPARTMENT_ID,DEPARTMENT_NAME
HAVING AVG(SALARY) >8000;
```

Query Result

SQL | All Rows Fetched: 6 in 0.151 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME	AVG(SALARY)
1	100	Finance	8601.333...
2	70	Public Relations	10000
3	90	Executive	19333.33...
4	110	Accounting	10154
5	20	Marketing	9500
6	80	Sales	8955.882...

Query Results Sorting

ORDER BY clause allows you to order the tuples in the result of a query.

Notes:

- In ORDER BY clause you can use column name, column alias, column number, aggregate function or mathematical expression.
- The default order is in ascending order of values.
- To reverse the ordering, use “DESC” keyword.

Q33: Retrieve all employees sorted by their first name.

```
SELECT EMPLOYEE_ID,FIRST_NAME,LAST_NAME
FROM EMPLOYEES
ORDER BY FIRST_NAME;
```

Query Result | SQL | Fetched 50 rows in 0.516 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	121	Adam	Fripp
2	196	Alana	Walsh
3	147	Alberto	Errazuriz
4	103	Alexander	Hunold
5	115	Alexander	Khoo
6	185	Alexis	Bull
7	158	Allan	McEwen
8	175	Alyssa	Hutton
9	167	Amit	Banda
10	187	Anthony	Cabrio
11	193	Britney	Everett

```
SELECT EMPLOYEE_ID,FIRST_NAME,LAST_NAME
FROM EMPLOYEES
ORDER BY 2;
```

Query Result | SQL | Fetched 50 rows in 0.092 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	121	Adam	Fripp
2	196	Alana	Walsh
3	147	Alberto	Errazuriz
4	103	Alexander	Hunold
5	115	Alexander	Khoo
6	185	Alexis	Bull
7	158	Allan	McEwen
8	175	Alyssa	Hutton
9	167	Amit	Banda
10	187	Anthony	Cabrio
11	193	Britney	Everett

Q34: Retrieve all employees' records sorted by their hire date, such that the newest employee should come first and the old one come last.

```
SELECT EMPLOYEE_ID,FIRST_NAME,LAST_NAME,HIRE_DATE
FROM EMPLOYEES
ORDER BY 4 DESC;
```

Query Result | SQL | Fetched 50 rows in 0.349 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	HIRE_DATE
13	135	Ki	Gee	12-DEC-07
14	113	Luis	Popp	07-DEC-07
15	155	Oliver	Tuvault	23-NOV-07
16	124	Kevin	Mourgos	16-NOV-07
17	148	Gerald	Cambrault	15-OCT-07
18	119	Karen	Colmenares	10-AUG-07
19	198	Donald	OConnell	21-JUN-07
20	182	Martha	Sullivan	21-JUN-07
21	178	Kimberely	Grant	24-MAY-07

Q35: Retrieve all employees sorted by their total salary.

Total salary = salary + salary * commission_pct

```
SELECT EMPLOYEE_ID,FIRST_NAME,SALARY+SALARY*NVL(COMMISSION_PCT,0)"TOTAL SALARY"
FROM EMPLOYEES
ORDER BY "TOTAL SALARY";
```

Query Result | SQL | Fetched 50 rows in 0.047 seconds

	EMPLOYEE_ID	FIRST_NAME	TOTAL SALARY
1	132	TJ	2100
2	128	Steven	2200
3	136	Hazel	2200
4	127	James	2400
5	135	Ki	2400
6	119	Karen	2500

Q36: Retrieve all employees sorted by their total salary in descending order, if there are two employees have the same total salary, then sort them by first name in ascending order, then by their last name.

```
SELECT FIRST_NAME,LAST_NAME,SALARY+SALARY*NVL(COMMISSION_PCT,0)"TOTAL SALARY"
FROM EMPLOYEES
ORDER BY 3 DESC,FIRST_NAME,2;
```

Query Result

All Rows Fetched: 107 in 0.16 seconds

	FIRST_NAME	LAST_NAME	TOTAL SALARY
1	Steven	King	24000
2	John	Russell	19600
3	Karen	Partners	17550
4	Lex	De Haan	17000
5	Neena	Kochhar	17000
6	Alberto	Errazuriz	15600
7	Lisa	Ozer	14375
8	Ellen	Abel	14300
9	Gerald	Cambrault	14300

Exercises

1. Write SQL statement to insert a new record into JOBS table with the following information:

   JOB_ID = C_ENG
   JOB_TITLE = Computer Engineer
   MIN_SALARY = 20000
   MAX_SALARY = 50000

2. Retrieve all employees who aren't working in any department.
3. Write SQL statement to retrieve the last name and salary for all employees whose salary is not in the range 5000 through 12000.
4. Write SQL statement to retrieve the last names of all employees who have both an "a" and an "e" in their last name.
5. Write SQL statement to retrieve the last name, salary, and commission for all employees who earn commissions. Sort data in descending order of salary and commissions.
6. Write SQL statement to retrieve all employee last names in which the third letter of the name is "a".
7. Write a query to retrieve employees whose job has minimum salary 4200 and maximum salary 9000.
8. Write a query to retrieve departments' names and cities for all departments that has more than 5 employees.

Lab 06-DML II

1. Write a query to get any employee who has salary greater than or equal to 10000.

```
use hr;
Select employee_id, first_name, last_name, salary
from employees
where salary >=10000;
```

100% 1:3

Result Grid | Filter Rows: Search | Edit: | Export/Import:

employee_id	first_name	last_name	salary
100	Steven	King	24000.00
101	Neena	Kochhar	17000.00
102	Lex	De Haan	21000.00
108	Nancy	Greenberg	12008.00
114	Den	Raphaely	11000.00
145	John	Russell	14000.00
146	Karen	Partners	13500.00

2. Write a query to find employee who has salary from 5K to 10K.

```
use hr;
Select employee_id, first_name, last_name, salary
from employees
where salary BETWEEN 5000 AND 10000;
```

100% 37:4

Result Grid | Filter Rows: Search | Edit: | Export/Import:

employee_id	first_name	last_name	salary
103	Alexander	Hunold	9000.00
104	Bruce	Ernst	6000.00
109	Daniel	Faviet	9000.00
110	John	Chen	8200.00
111	Ismael	Sciarra	7700.00
112	Jose Manuel	Urman	7800.00
113	Luis	Popp	6900.00
120	Matthew	Weiss	8000.00

3. Write a query to get any employee who is from department 'Executive'*.

```
1  use hr;
2  Select employee_id, first_name, last_name, salary
3  from employees
4  where department_id = 90;
```

100% 26:4

Result Grid | Filter Rows: Search | Edit: | Export/Import:

employee_id	first_name	last_name	salary
100	Steven	King	24000.00
101	Neena	Kochhar	17000.00
102	Lex	De Haan	21000.00

4. Write a query to get all employee who is working on department 50, 70, 90.
5. Write a query to find employee whose first_name starts with an 'O'
6. Write a query to find employee whose name contains the pattern 'la'
7. Write a query to find employee whose last_name starts with k, ends with o and contains any characters between e.g., Kano, Kenjiro, Kentaro.
8. Write a query to display 3 employees start from record number 7 (which is the 8th record)